Encrypted Cloud Vault with SSE

A REPORT

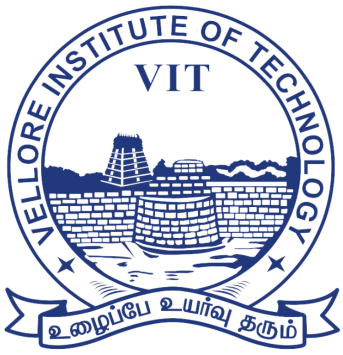
submitted by

Sowjhanya R (24BYB1174)
Sakthi Shivram (24BYB1168)

Submitted for the course Cryptography and Network Security (BCSE309L) in partial fulfilment of the requirements of B. Tech. School of Computer Science and Engineering.

B. Tech. Computer Science and Engineering

School of Computer Science and Engineering (SCOPE)



VIT®
CHENNAI
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Faculty
Dr. Nithyanandam P
Professor and Associate Dean, SCOPE

October 2026

Abstract: Cloud storage is cheap and easy to use, but the people who run the servers can read whatever is stored there. Encrypting files before uploading them fixes this, but then the server cannot search the files, and the user has to download everything just to find one document. In this project we built an encrypted cloud vault that keeps files confidential and still supports keyword search, using Searchable Symmetric Encryption (SSE). The system runs as two separate Flask processes. A trusted client does all the cryptography and serves the web interface, and an untrusted cloud service stores only ciphertext and an encrypted index. A vault passphrase is turned into a master key with Argon2id, and HKDF-SHA256 splits it into a file key and an index key. File contents and filenames are encrypted with AES-256-GCM. The search index follows the basic construction of Cash et al., where HMAC-SHA256 produces index labels and search tokens, so the cloud can find matching entries without learning the keyword. Multi-keyword (AND) queries are answered by intersecting single-keyword results on the client. The current version also has user accounts, an administrator role, an audit log, an Encryption Transparency Lab, a Cloud Storage Inspector and a tamper-detection test. All 406 automated tests passed in our final run. We also describe what the cloud can still learn, such as repeated searches, access patterns and file sizes.

Keywords: searchable symmetric encryption; encrypted cloud storage; AES-256-GCM; Argon2id; HKDF-SHA256; HMAC-SHA256; leakage

I. INTRODUCTION

Many students and small organisations keep their documents on cloud storage. It is convenient because the files can be reached from anywhere, and the provider handles backups. The problem is trust. The provider, a dishonest employee, or an attacker who breaks into the server can read any file that is stored in plain form. Encryption on the server side does not fully help, because the provider also holds the key.

The usual answer is client-side encryption: the user encrypts each file before it leaves their machine, and the server only ever sees ciphertext. This protects confidentiality, but it breaks a basic feature. A server that only has random-looking bytes cannot answer a question like "which of my files contain the word *invoice*?" The user would need to download and decrypt every file to search them, which does not scale.

Searchable Symmetric Encryption (SSE) is a family of techniques for exactly this problem. The idea was introduced by Song, Wagner and Perrig [1], and later work by Curtmola et al. [2] gave formal security definitions and efficient index-based schemes. The client builds an encrypted index over the keywords of its documents and uploads it with the encrypted files. To search, the client sends a short token derived from the keyword and a secret key. The server uses the token to find the matching index entries, but it does not learn the keyword itself. Efficient SSE schemes do leak some information, for example which entries match a query, and this trade-off is an important part of our report.

In this project we implemented a working encrypted cloud vault based on the basic SSE construction (called Π_{bas}) of Cash et al. [3]. The main goals were:

- keep file contents and filenames secret from the cloud, and detect any change to stored data;
- derive all keys from a passphrase without ever storing a key;
- let the cloud answer single-keyword and multi-keyword (AND) searches without seeing keywords;
- support several users, each with an isolated vault, plus an administrator who cannot read files;
- make the system easy to inspect, by showing the actual ciphertext and the actual search messages in the interface; and
- state clearly what the cloud can still learn, instead of claiming perfect secrecy.

The rest of the report is organised as follows. Section II reviews the standards and research we relied on. Section III describes the architecture and Section IV the workflows. Section V explains the cryptographic algorithms as they are implemented in the code. Section VI shows results from a demonstration vault, Section VII covers testing and the security evaluation, and Sections VIII and IX give the limitations, future work and conclusion.

II. RELATED WORK

2.1 Cryptographic Standards Used

Our system does not invent any new primitive. It combines standard building blocks. AES is specified in FIPS 197 [4], and the Galois/Counter Mode (GCM) that we use for authenticated encryption is specified in NIST SP 800-38D [5]. GCM gives confidentiality and also an authentication tag, so a modified ciphertext is rejected during decryption. For turning a human-chosen passphrase into a key we use Argon2id, standardised in RFC 9106 [6]. Argon2 is memory-hard, which makes large-scale password guessing on GPUs expensive. HKDF, defined in RFC 5869 [7], expands one strong key into several independent keys using different “info” labels. HMAC is defined in RFC 2104 [8] and FIPS 198-1 [9], and we use it with SHA-256 from FIPS 180-4 [10] as a pseudorandom function (PRF) for search tokens and index labels.

2.2 Searchable Symmetric Encryption

The first practical SSE scheme by Song et al. [1] encrypted each word of a document in a special way, so the server had to scan every word of every document during a search. Curtmola et al. [2] moved to an inverted-index design, where search time depends on the number of matches, and defined security in terms of the information leaked to the server. Cash et al. [3] presented Π_{bas}, a simple and efficient scheme that stores the index as a flat dictionary of label–value pairs. For a keyword w, the labels are produced by a PRF from a keyword-specific key and a counter, and each value is an encrypted document identifier. Our index follows this design. In an earlier paper, Cash et al. [11] proposed OXT, which supports conjunctive and Boolean queries while hiding more from the server than running separate single-keyword searches. OXT needs extra structures and public-key style operations, so we did not implement it. Instead, our AND search runs one Π_{bas} search per keyword and intersects the results on the client, which is simpler but leaks more.

2.3 Leakage and Attacks

Efficient SSE schemes accept some leakage, and research has shown that this leakage can be harmful. Islam, Kuzu and Kantarcioglu [12] showed that the access pattern (which encrypted documents match each query), combined with some background knowledge, can be used to guess the queried keywords. Cash et al. [13] studied leakage-abuse attacks in more detail and showed when such attacks work well and when they do not. Zhang, Katz and Papamanthou [14] showed file-injection attacks, where an attacker who can make the user store chosen files learns queried keywords very quickly. These attacks are much more effective against schemes that are not forward private. Bost [15] presented Σοφος, a forward-private SSE scheme, in which tokens issued earlier cannot be used to match files that are added later. Our project uses a static index without forward privacy, so these papers mainly help us explain the limits of our design.

Approach	Ref.	Status in our project
AES-256-GCM authenticated encryption	[4], [5]	Implemented for file contents, filenames, index values and the verifier
Argon2id passphrase-based key derivation	[6]	Implemented (t = 3, m = 64 MiB, p = 4)
HKDF-SHA256 key separation	[7]	Implemented (two sub-keys with versioned labels)
HMAC-SHA256 as a PRF	[8]–[10]	Implemented for tokens and index labels
Index-based SSE, Π_{bas}	[2], [3]	Implemented as a static index, fully rebuilt on each change
OXT conjunctive search	[11]	Not implemented; AND is done by intersecting results on the client
Forward-private SSE (Σοφος)	[15]	Not implemented; listed as future work
Leakage-abuse and file-injection attacks	[12]–[14]	Discussed in Section VII; not mitigated

Table I Related approaches and their status in this project

III. PROPOSED SYSTEM AND ARCHITECTURE

3.1 Threat Model

We treat the cloud service as *honest-but-curious*. It stores what it is given and answers requests correctly, but it may look at everything it stores and every request it receives. The goal is that the cloud learns as little as possible about file contents, filenames and search keywords. We also want to detect if stored data is modified. The trusted client runs on the user's own machine and is assumed to be safe. If the client machine itself is compromised, an attacker could capture passphrases as they are typed, and our design does not protect against that. Inside the client there are also other users and an administrator, and the system tries to keep each user's files away from them.

3.2 Two-Process Architecture

The system is split into two Flask applications that run as separate operating-system processes (Fig. 1). The trusted client (`client/client_app.py`) listens on port 5050 on macOS (5000 by default) and serves the browser interface from the `web/` folder. The untrusted cloud (`cloud/cloud_app.py`) listens on port 5001 and keeps its data in a SQLite database. The browser never talks to the cloud directly and does no cryptography; every request goes to the trusted client under `/api/*`. The client then talks to the cloud over HTTP with JSON, where binary fields are sent as base64. The cloud code does not import any of the client's cryptography or key-management modules. For its search loop it computes HMAC-SHA256 itself with the Python standard library.

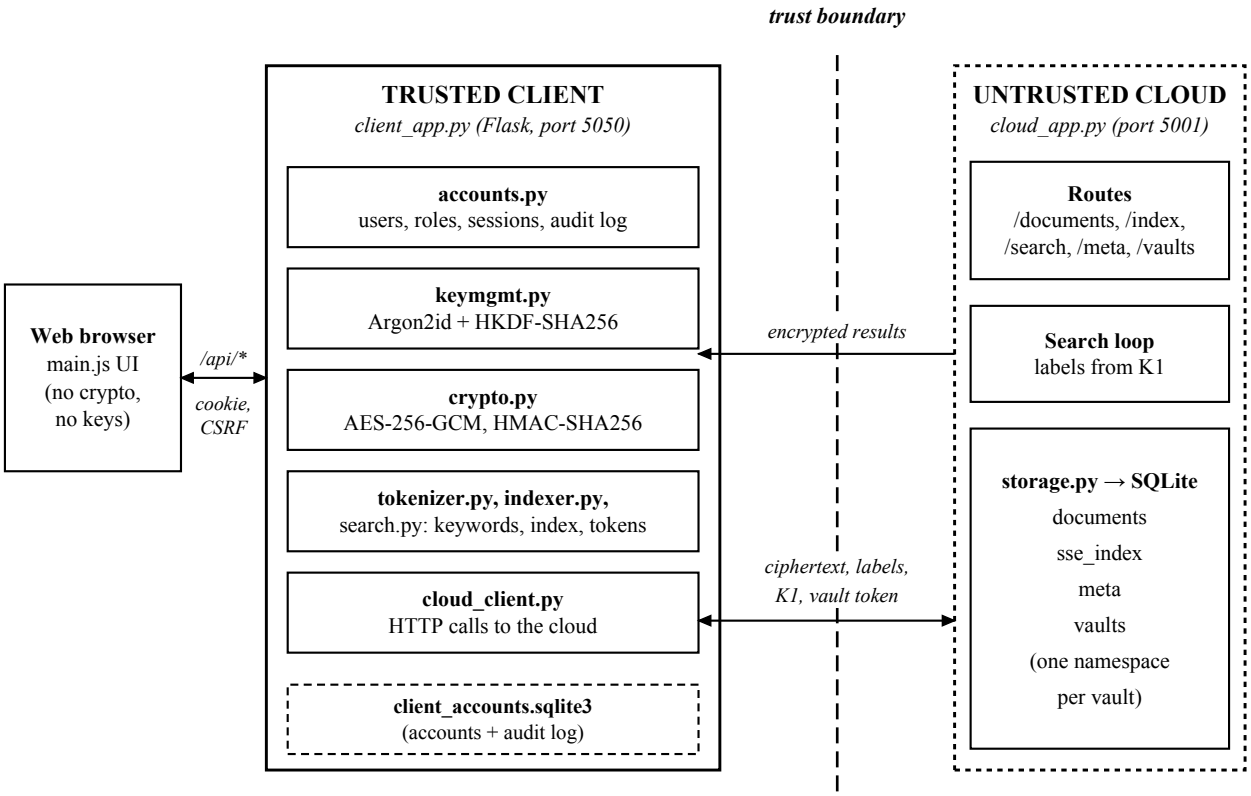


Fig. 1 System architecture and trust boundary

3.3 Main Modules

Table II lists the main source files and what each one does. Keeping the cryptographic code in small modules made it easier to test each part on its own.

Module	Side	Responsibility
`client/crypto.py`	Client	AES-256-GCM encrypt/decrypt with random 96-bit nonces; HMAC-SHA256 PRF; HKDF-SHA256
`client/keymgmt.py`	Client	Argon2id master key, HKDF sub-keys K_file and K_index, key wiping on lock
`client/tokenizer.py`	Client	One shared keyword extractor for both indexing and searching
`client/indexer.py`	Client	Builds the flat Π_{bas} index of (label, value) pairs
`client/search.py`	Client	Query normalisation, search tokens, decryption of search results
`client/textextract.py`	Client	Extracts text from UTF-8 files, PDFs (pypdf) and ZIP archives
`client/accounts.py`	Client	Accounts, Argon2id password hashes, roles, sessions, audit log
`client/cloud_client.py`	Client	HTTP client for the cloud API, sends the vault id and bearer token
`client/client_app.py`	Client	Flask routes for the UI, upload, search, Lab, Inspector and admin pages
`cloud/cloud_app.py`	Cloud	Flask routes for storage, index replacement, search loop and metadata
`cloud/storage.py`	Cloud	SQLite schema, per-vault namespaces and schema migration

Table II Main modules of the implementation

3.4 Data Stored by the Cloud

Table III shows the four tables in the cloud database. Every table carries a `vault_id` column, so each user's data lives in its own namespace. None of the columns hold a key, a plaintext filename, a keyword or any plaintext file content.

Table	Columns	What is stored
`documents`	doc_id, enc_name, nonce, ciphertext, tag, size, uploaded_at, vault_id	One row per file. The id is random, the name and content are AES-GCM ciphertext
`sse_index`	vault_id, label, value	Encrypted index: HMAC label (hex) and AES-GCM encrypted document id
`meta`	vault_id, key, value	Non-secret KDF data: kdf_salt and kdf_verifier
`vaults`	vault_id, token_hash, created_at	Registered vaults with SHA-256 of each vault's access token

Table III Tables in the untrusted cloud database

IV. SYSTEM DESIGN AND WORKFLOW

4.1 Accounts, Sessions and Roles

The current version supports several users. A user first creates an account with a username and an account password of at least eight characters. The account password is stored only as an Argon2id password hash (argon2-cffi's `PasswordHasher`) in `client_accounts.sqlite3` on the trusted side. It is only used to log in and never derives a key. Each new account also gets a random 16-byte vault id and a random 32-byte cloud access token. The cloud stores only the SHA-256 hash of the token and compares it in constant time on every request that names a vault.

Sessions use an HttpOnly and SameSite=Strict cookie that holds only a random session id. A session expires after 30 minutes of inactivity or 8 hours in total. Every state-changing request must carry a CSRF token in the `X-CSRF-Token` header, and logins for a username are throttled after five failures within 15 minutes. There are two roles, *user* and *admin*. No administrator exists by default. The first administrator is created once with a setup token printed in the client terminal, and normal registration can never create an administrator.

4.2 Vault Unlock

Logging in does not open the vault. The user must also enter a separate *vault passphrase* (Fig. 2). On the first unlock, the client creates a random 16-byte salt, derives the keys and stores the salt and an encrypted *verifier* in the cloud `meta` table. On later unlocks, the client fetches the salt, derives the keys again, and tries to decrypt the verifier. If the AES-GCM tag check fails, the passphrase is wrong and the request is rejected before any file or index is touched. The keys are kept only in the memory of that server-side session and are wiped when the user locks the vault, logs out, the session expires or the account is deactivated.

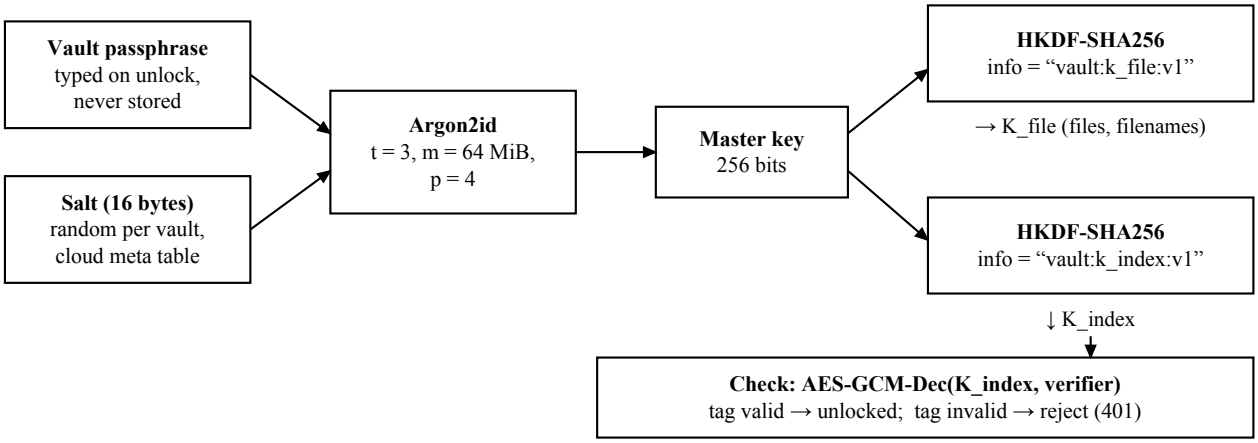


Fig. 2 Key derivation and vault unlocking

4.3 Upload Workflow

Fig. 3 shows the upload steps. The client first checks that the file is UTF-8 text, a PDF with a text layer, or a ZIP archive of those, with a maximum size of 16 MB. It then gives the file a random document id and encrypts the content and the filename separately under K_{file}, each with a new random nonce. The encrypted record is pushed to the cloud. After that, the client rebuilds the whole search index: it fetches and decrypts every document in the vault, extracts keywords, builds a new index and sends it to `/index/replace`. The cloud deletes the old index rows and inserts the new ones inside one database transaction. If any stored document fails to decrypt during the rebuild, the rebuild is aborted and the old index is left unchanged.

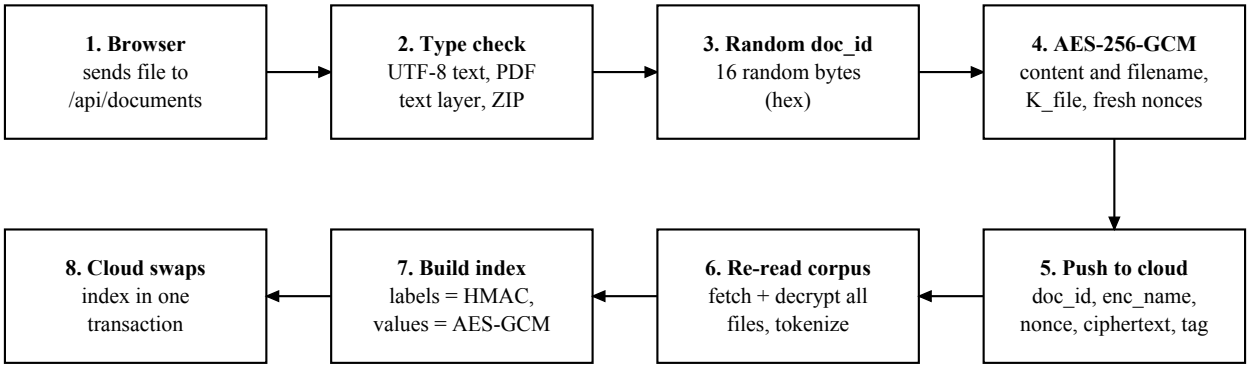


Fig. 3 Upload, encryption and index rebuild workflow

4.4 Search Workflow

Fig. 4 shows a search. The query goes through the same tokenizer that was used for indexing, so the keywords match exactly. For each keyword the client derives a token K_1 and sends only that token to the cloud. The cloud regenerates the labels for counters 0, 1, 2, and so on, and returns the stored values until the first label that is missing. The client derives K_2 locally, decrypts the returned values into document ids, intersects the id sets if there were several keywords, and finally fetches and decrypts the filenames to show the results. K_2 is never sent to the cloud, because the cloud never needs to decrypt anything.

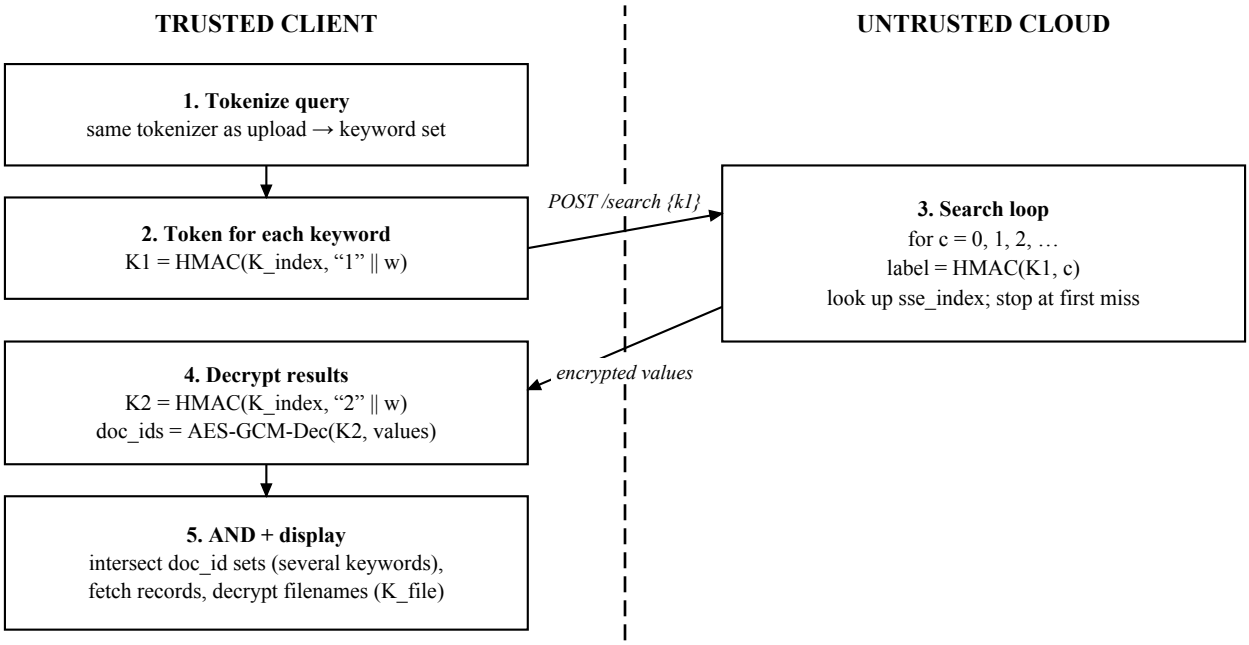


Fig. 4 Encrypted keyword search workflow

4.5 Download, Deletion and Passphrase Change

To view or download a file, the client fetches the record, joins nonce, ciphertext and tag, and decrypts it with K_{file}. For deletion the order is reversed compared with upload. The index is first rebuilt without the document, and only then is the ciphertext deleted. If the rebuild fails, the file still exists and the request can simply be retried, instead of leaving index entries that point to a deleted file.

The vault passphrase can be changed from the Security & System page. The client checks the current passphrase against the verifier, derives new keys from a new salt, re-encrypts every document and filename, rebuilds the index, and updates the salt and verifier as the last step. This is not atomic. If it is interrupted halfway, the vault can contain a mix of old and new ciphertexts, and the error message explains how to finish by retrying with the same new passphrase.

4.6 Administrator Functions and Audit Log

An administrator has no vault and no keys. The admin pages show account metadata, counts of events such as failed logins or rejected tamper tests, an audit log, and a Cloud Storage Inspector that displays the ciphertext records of a chosen user's vault. An administrator can deactivate an account, which also ends that user's live sessions. The audit log records who did what, to which opaque id, and the outcome. By design it never stores passwords, keys, file contents, plaintext filenames or search keywords; a search event only records how many keywords and results there were.

V. CRYPTOGRAPHIC ALGORITHMS AND IMPLEMENTATION

5.1 Key Derivation

The master key MK is derived from the vault passphrase *pw* and the 16-byte salt with Argon2id through argon2-cffi's low-level `hash_secret_raw` function. The parameters (t = 3 passes, m = 64 MiB, p = 4 lanes, 256-bit output) are the same as the second recommended option in RFC 9106 [6]. HKDF-SHA256 then derives two 256-bit sub-keys with different info labels. The salt argument of HKDF is left empty because the input is already a uniformly strong key, and the domain separation comes from the labels.

$$MK = Argon2id(pw, salt;\ t = 3,\ m = 64\ MiB,\ p = 4) \quad (1)$$

$$K_{file} = HKDF(MK, \text{“vault:k_file:v1”}),\ K_{index} = HKDF(MK, \text{“vault:k_index:v1”}) \quad (2)$$

Because Argon2id with the same inputs always gives the same key, nothing needs to be stored to unlock the vault later except the salt. To detect a wrong passphrase, a fixed string is encrypted once under K_{index} and stored as the verifier:

$$V = AES\text{-}GCM\text{-}Enc(K_{index}, \text{“vault-verifier:v1”}) \quad (3)$$

The `KeySet` object that holds K_{file} and K_{index} is never serialised. Its `wipe()` method replaces the stored keys with zero bytes on lock and logout. We note in the code and here that this is only best-effort in Python, because `bytes` objects are immutable and copies may remain in memory.

5.2 File and Filename Encryption

Each file M and each filename are encrypted with AES-256-GCM using the `AESGCM` class of the Python `cryptography` library. A fresh 96-bit nonce N is generated with `os.urandom` on every call and the caller cannot supply one, so a nonce cannot be reused by mistake. The library appends a 128-bit tag T to the ciphertext CT. The stored blob is:

$$C = AES\text{-}GCM\text{-}Enc(K_{file}, M) = N \,||\, CT \,||\, T,\ |C| = |M| + 12 + 16 \quad (4)$$

GCM works like a stream cipher, so the ciphertext has the same length as the plaintext and the total overhead is always 28 bytes. The cloud keeps N, CT and T in separate columns. During decryption the library recomputes the tag. If even one bit of the nonce, ciphertext or tag was changed, it raises `InvalidTag`, which our code turns into a single `DecryptionError` without saying which part failed, so no partial plaintext is ever returned. We also note that NIST SP 800-38D [5] limits how many messages can be encrypted under one key with random 96-bit nonces; a personal vault stays far below that limit, but a very large deployment would need to count encryptions or rotate keys.

5.3 Keyword Extraction

The tokenizer lowercases the text, takes maximal runs of ASCII letters and digits as tokens, drops tokens shorter than two characters and a fixed list of 60 English stop words, and removes duplicates. There is no stemming, so a search for *run* does not match *running*. The same `tokenize()` function is used for indexing and for search queries. If the two paths used different rules, searches would silently miss documents.

5.4 Encrypted Index Construction

For every keyword w in the vault, the client derives two keyword keys from K_{index}. Here F is HMAC-SHA256 and “1” and “2” are single domain-separation bytes:

$$K_1 = F(K_{index}, \text{“1”} \,||\, w),\ K_2 = F(K_{index}, \text{“2”} \,||\, w) \quad (5)$$

If DB(w) is the list of document ids that contain w, sorted by id, then entry number c of that list becomes one row of the index. The counter is encoded as 8 bytes, big-endian:

$$L_c = F(K_1, c),\ v_c = AES\text{-}GCM\text{-}Enc(K_2, id_c),\ c = 0, 1, \ldots, |DB(w)| - 1 \quad (6)$$

All rows of all keywords go into one flat table, so the table itself does not show which rows belong to the same keyword. Labels are deterministic, but each value uses a new random nonce, so rebuilding the index gives different value bytes even for unchanged postings. Algorithm 1 summarises the construction.

Algorithm 1: Build the encrypted index (indexer.build_index)

Input: K_{index}; map from doc_id to its keyword set W(doc_id)

Output: list of (label, value) pairs

1. postings ← empty map; for each doc_id in sorted order, for each w in W(doc_id): append doc_id to postings[w]
2. for each keyword w in postings:
3. $K_1 \leftarrow F(K_{index}, \text{“1”} \| w)$; $K_2 \leftarrow F(K_{index}, \text{“2”} \| w)$
4. for c = 0, 1, … and each doc_id in postings[w]:
5. label ← hex($F(K_1$, c as 8 bytes)); value ← AES-GCM-Enc(K_2, doc_id)
6. add (label, value) to the output
7. return output (the client then calls POST /index/replace)

5.5 Search Token Generation and Cloud Search

In the original Π_{bas} of Cash et al. [3], the client sends both K_1 and K_2, and the server decrypts the document ids itself. Our implementation changes this: the client sends only $T_w = K_1$, and K_2 stays on the client, because the cloud never needs to decrypt anything. The cloud runs the loop in Algorithm 2 and returns the set of values it found:

$$R(w) = \{ v_c : F(T_w, c) \text{ is in the index; } c = 0, 1, 2, \ldots \text{ until the first miss } \} \quad (7)$$

Algorithm 2: Cloud search loop (cloud_app.run_search_loop)

Input: token K_1; vault_id **Output:** list of encrypted values

1. n ← number of index rows in this vault; results ← [ ]; c ← 0
2. loop: if c > n then raise an error (corrupted index or bug)
3. label ← hex(HMAC-SHA256(K_1, c as 8 bytes)); value ← lookup(vault_id, label)
4. if value is missing then return results
5. append value to results; c ← c + 1

The client then derives K_2 for the same keyword and decrypts every returned value. If any value fails the GCM check, the whole search is rejected with an error message about possible tampering, instead of silently returning fewer results.

5.6 Conjunctive (AND) Search

A query with several keywords means AND. The client runs one independent search per keyword in sorted order and intersects the decrypted id sets:

$$DB(w_1 \text{ AND } \ldots \text{ AND } w_q) = DB(w_1) \cap DB(w_2) \cap \ldots \cap DB(w_q) \quad (8)$$

If one keyword returns no documents, the remaining keywords are not sent because the intersection is already empty. The intersection happens on the trusted client, but the cloud still sees one token per keyword arriving together, and the result count of each one.

5.7 Summary of Primitives

Primitive	Parameters	Library	Used for
AES-256-GCM	256-bit key, 96-bit random nonce, 128-bit tag	cryptography (AESGCM)	File contents, filenames, index values, verifier
Argon2id	t = 3, m = 64 MiB, p = 4, 16-byte salt, 32-byte output	argon2-cffi	Vault passphrase → master key
Argon2id hash	library defaults, salt inside hash string	argon2-cffi (PasswordHasher)	Account password storage
HKDF-SHA256	no salt, info "vault:k_file:v1" / "vault:k_index:v1"	cryptography (HKDF)	Master key → K_file, K_index
HMAC-SHA256	256-bit keys	Python hmac + hashlib	K1, K2, index labels, cloud search loop
SHA-256	–	Python hashlib	Hash of vault access tokens; tamper test checksums

Table IV Cryptographic primitives in the implementation

VI. RESULTS AND DISCUSSION

6.1 Demonstration Setup

All screenshots in this section were captured from a **demo vault**. We ran an isolated copy of the project with fresh temporary databases, created demo-only accounts (`demo_alice`, `demo_bob` and `demo_admin`) and uploaded the eight short sample text files included with the project in `data/sample_corpus` (4,129 bytes in total). No real documents or credentials were used. The browser interface was switched to its built-in light theme for printing. The services ran on a MacBook with an Apple M4 Pro chip and macOS 26.6.2, using Python 3.9.6, Flask 3.1.3, cryptography 50.0.2 and argon2-cffi 25.1.0. We did not measure performance for this report, so no timing figures are given.

6.2 Login and Vault Unlock

Fig. 5 shows the login page. The left side summarises the cryptography, and the right side has tabs for logging in and creating an account. After the first login, the user is asked to choose a vault passphrase, which needs to be typed twice. The page warns that the passphrase is never stored and that files cannot be recovered without it, even by an administrator. While the keys are being derived, the page shows a message that the Argon2id delay is deliberate. While the vault is locked, the file list, upload, search and Lab pages ask for the passphrase first, but the dashboard, Cloud Storage Inspector, Activity History and Security & System pages still open, because they do not need a key.

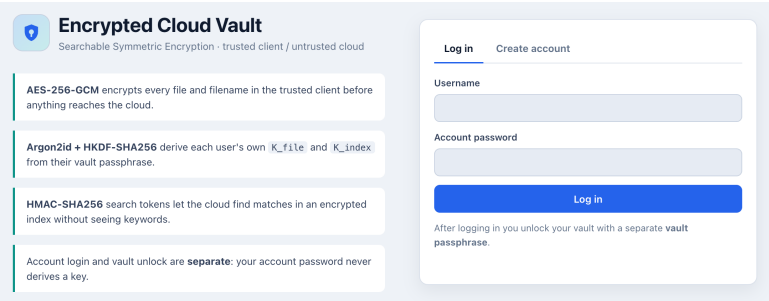


Fig. 5 Login page of the trusted client (demo vault)

6.3 Upload and Dashboard

Fig. 6 shows the upload page after encrypting `security_review.txt`. The progress list shows that the file was sent to the trusted client, encrypted with AES-256-GCM, pushed to the cloud, and that the index was rebuilt. After all eight sample files were uploaded, the dashboard in Fig. 7 reported 8 encrypted files, 4.0 KB of ciphertext plus 361 B of encrypted names, and 400 index entries. The ciphertext total matches the plaintext total of 4,129 bytes, which is what we expect from GCM. The 400 entries are the (keyword, file) pairs produced by the tokenizer for this corpus.

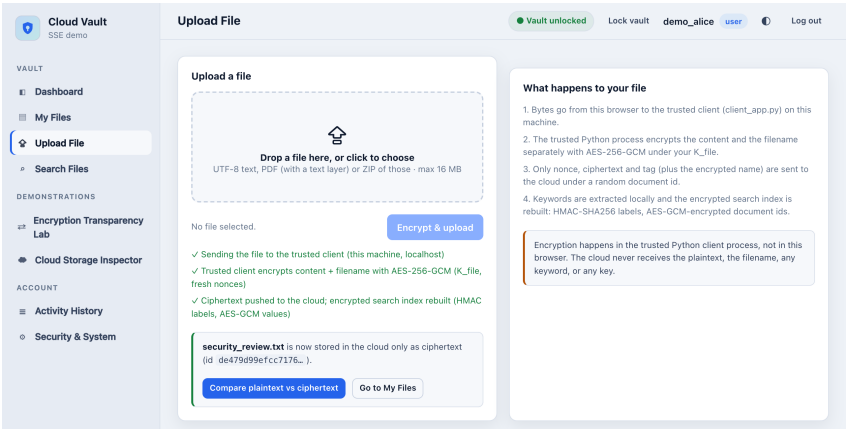


Fig. 6 Upload page after encrypting and uploading a sample file

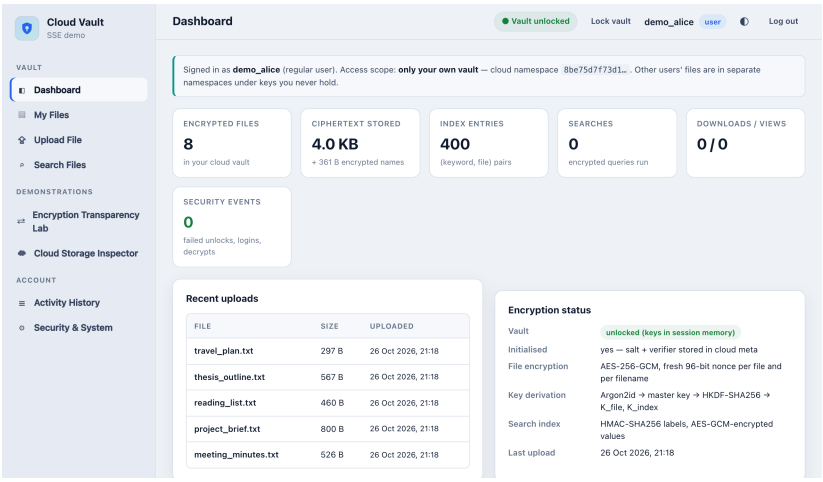


Fig. 7 User dashboard after uploading the eight sample files

6.4 Single-Keyword Search

Searching for *encryption* returned four files: `reading_list.txt`, `thesis_outline.txt`, `security_review.txt` and `project_brief.txt` (Fig. 8). This agrees with the keyword table in the sample corpus README, which was also made with the project's tokenizer. The step-by-step trace below the results is produced from real values of this query. It shows the 32-byte token K_1, the exact request body `{"k1": ...}` that went to the cloud, the four matching rows, and the four encrypted values of 60 bytes each. Each value is a 32-character document id plus the 28 bytes of GCM overhead. The cloud request contains no keyword, filename or key.

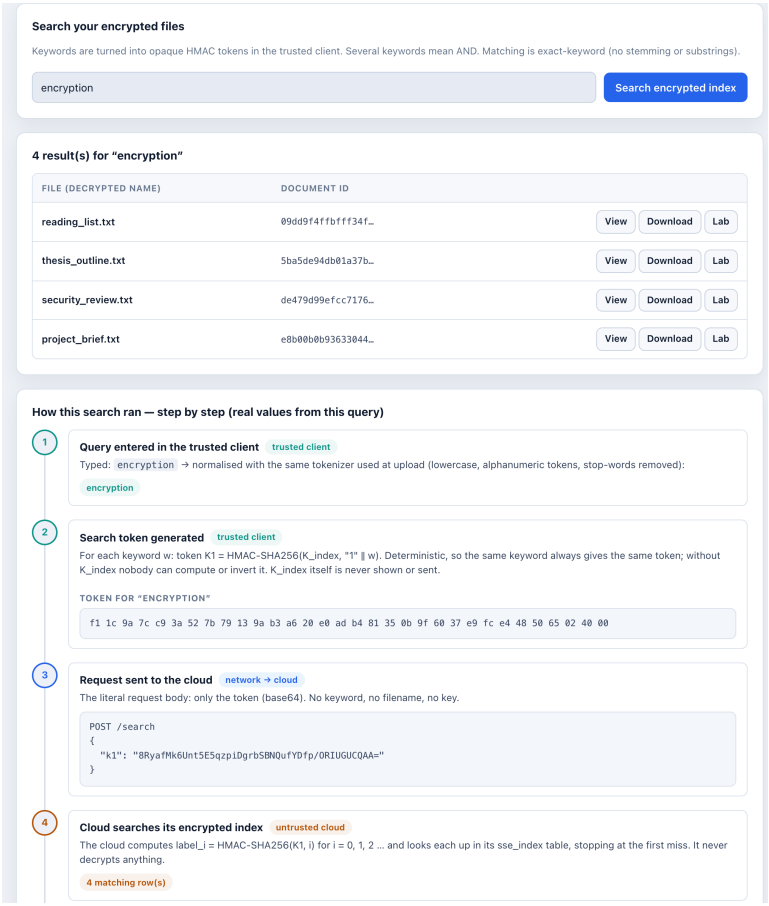


Fig. 8 Single-keyword search for “encryption” with the step-by-step trace

6.5 Conjunctive Search

For an AND query we searched *deadline benchmark* (Fig. 9). The client sent two separate `/search` requests, one token each. The cloud found four rows for *benchmark* and three for *deadline*. After decryption, the client intersected the two sets and two files remained, `meeting_minutes.txt` and `project_brief.txt`. The figure also makes the leakage visible: the cloud saw both tokens and both match counts, even though only two files were in the final answer.

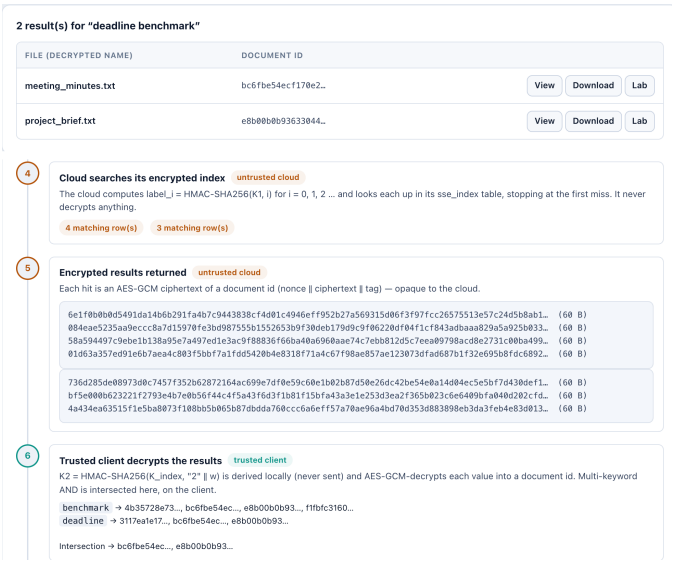


Fig. 9 Conjunctive search: two cloud searches, intersection on the client

6.6 Encryption Transparency Lab

The Encryption Transparency Lab places the plaintext view from the trusted client next to the record that the cloud API returns for the same file (Fig. 10). For `travel_plan.txt` the original file is 297 bytes, the stored ciphertext column is also 297 bytes, and the full blob with nonce and tag is 325 bytes, which is the expected 28-byte overhead. The 15-byte filename becomes a 43-byte encrypted blob. The page also checks that the first 32 plaintext bytes do not appear anywhere in the stored ciphertext. Only key names are shown, never key bytes.

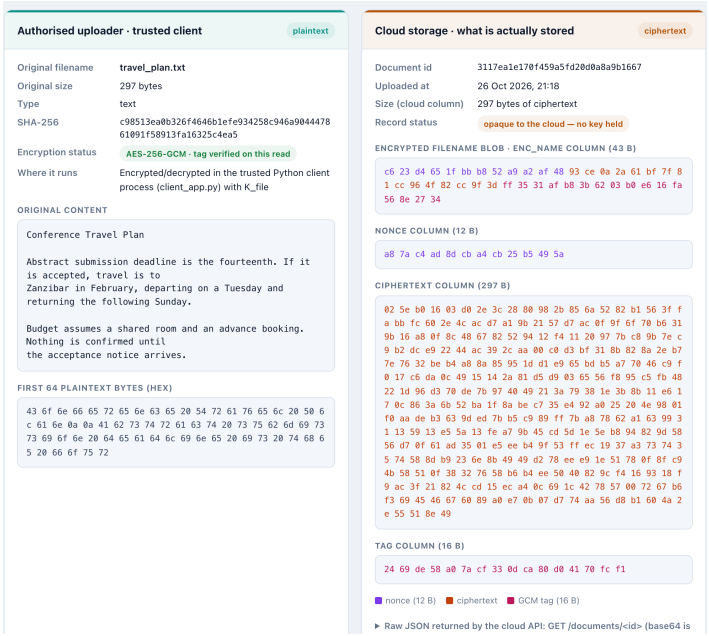


Fig. 10 Encryption Transparency Lab: plaintext in the trusted client vs. the cloud record

6.7 Cloud Storage Inspector

The Cloud Storage Inspector shows the vault exactly as the cloud stores it (Fig. 11). The document table has only random ids, sizes, timestamps and encrypted name prefixes. The index table shows 64-character HMAC labels and 60-byte encrypted values, with no column linking a row to a keyword. The page works even when the vault is locked, because no key is needed to display ciphertext. The same page also shows what is *not* hidden: the number of documents, their sizes, and the total number of index rows.

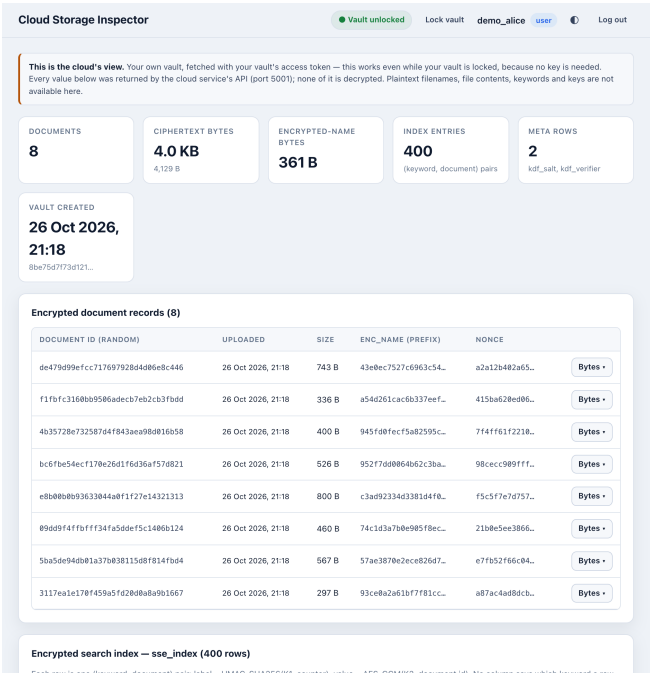


Fig. 11 Cloud Storage Inspector showing the stored ciphertext and index rows

6.8 Tamper Detection

The tamper-detection test copies an encrypted record into memory, flips one bit and tries to decrypt it with the real decryption code. In Fig. 12 we tested a copy of `security_review.txt`. Its blob is 771 bytes (743 plaintext bytes plus 28), and bit 5 of byte 499 of the ciphertext region was flipped at random. The unmodified blob decrypted correctly, while the modified copy was rejected with `DecryptionError` caused by `InvalidTag`. The record was fetched again from the cloud afterwards, and its SHA-256 matched the original, which shows that the test did not change stored data. The event was recorded in the audit log with a *demo* tag.

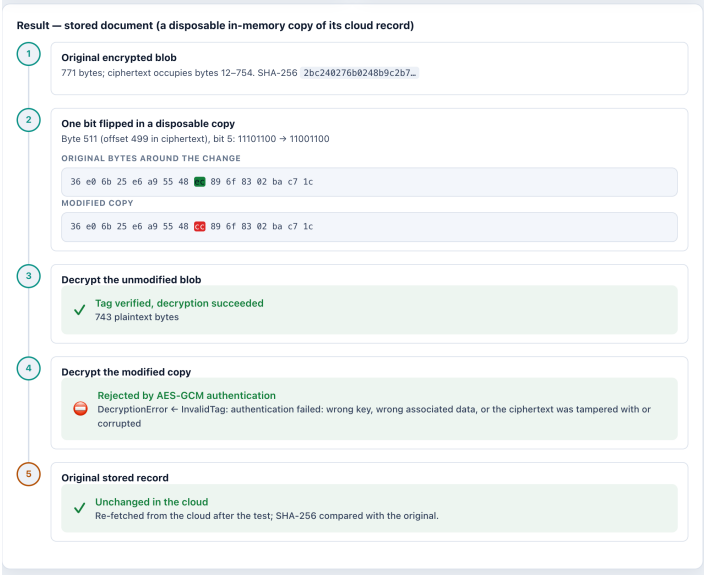


Fig. 12 Tamper-detection test: a one-bit change is rejected by AES-GCM

6.9 User Isolation and Administrator View

We then registered a second demo user, `demo_bob`, with a separate vault passphrase, and searched for the same keyword. Fig. 13 shows zero results and zero matching rows. The second user's token is derived from a different K_{index}, and the requests are scoped to that user's own vault id, so the index rows of `demo_alice` are not reachable.

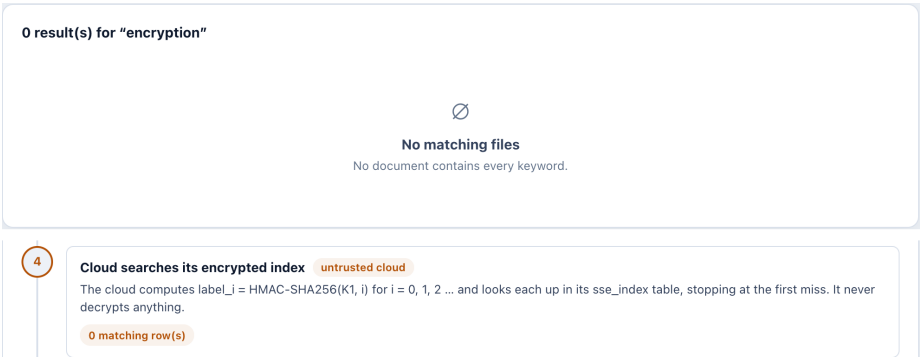


Fig. 13 Search by a second user (demo_bob) for the same keyword returns nothing

Finally we logged in as the administrator. The Admin Dashboard (Fig. 14) shows 3 accounts, 8 encrypted files, 400 index entries, 3 searches and 1 rejected tamper test, all counted from the audit log and the cloud's statistics. The Audit Log (Fig. 15) lists each event with actor, outcome and opaque target. Search events only say “1 keyword(s), 4 result(s)” or “2 keyword(s), 2 result(s)”. The query text and filenames are not recorded.

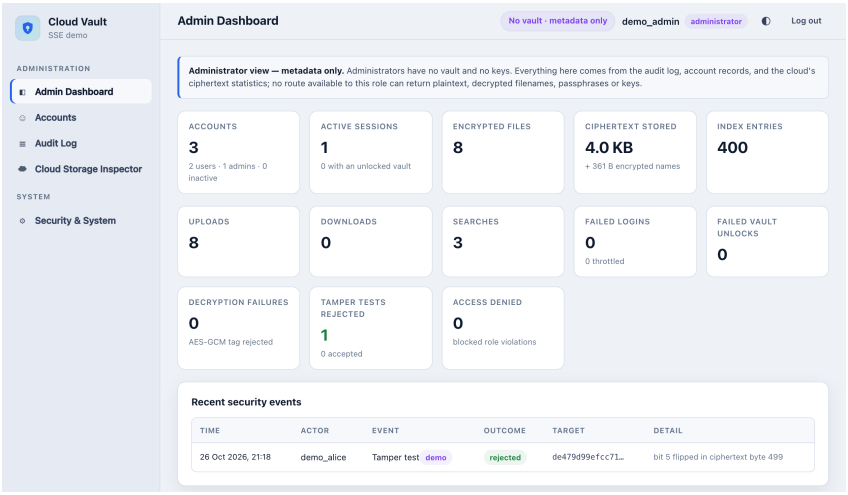


Fig. 14 Admin Dashboard: metadata and event counts only

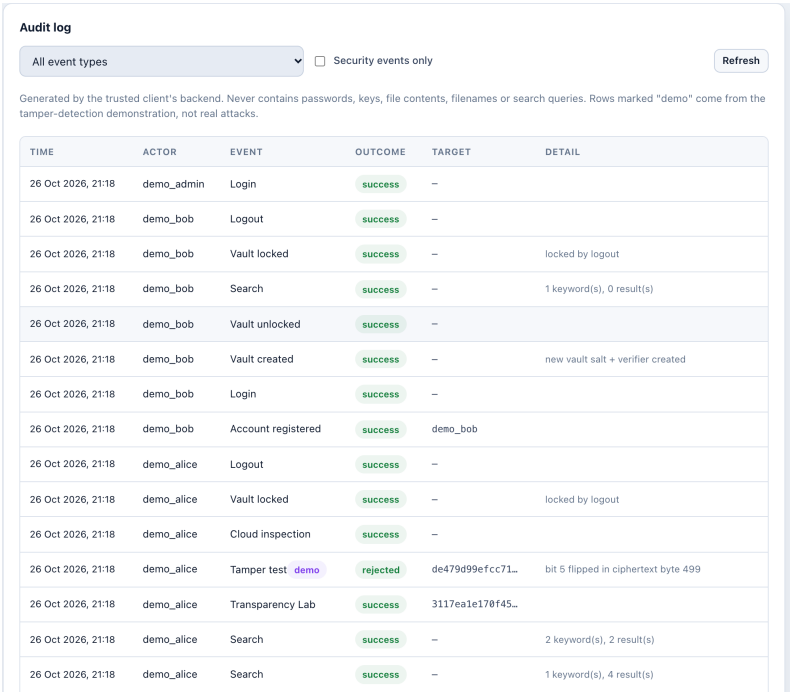


Fig. 15 Audit log viewed by the administrator (no keywords, filenames or keys)

VII. TESTING AND SECURITY EVALUATION

7.1 Automated Test Results

The project has an automated pytest suite in the `tests/` folder. For this report we ran it on a separate copy of the project, in a fresh virtual environment installed from `requirements.txt`, on the machine described in Section 6.1 (pytest 8.4.2). The result was **406 passed, 1 warning in 29.17 s**. The single warning came from the urllib3 library, which reported that the system Python was built with LibreSSL instead of OpenSSL; it is not related to the project code. Table V shows the number of tests per file.

Test file	Tests	Main areas covered
test_client_app.py	80	Client routes: unlock, upload, view, download, delete, search, passphrase change
test_multiuser_security.py	69	Accounts, sessions, CSRF, roles, isolation, admin, Lab, Inspector, tamper test
test_cloud_app.py	43	Cloud HTTP routes, index replacement and the search loop
test_crypto.py	36	AES-GCM, HMAC PRF and HKDF behaviour
test_storage.py	32	SQLite storage layer: documents, index rows and metadata
test_search.py	29	Query normalisation, tokens, decryption of results
test_textextract.py	28	Text, PDF and ZIP extraction rules
test_indexer.py	23	Index construction, labels, counters, fresh nonces
test_integration.py	21	End-to-end round trips through the cloud service; no key material in the database or requests
test_tokenizer.py	18	Keyword extraction and stop words
test_cloud_client.py	16	HTTP client for the cloud API
test_security_demonstrations.py	11	Leakage and attack demonstrations D1–D9 on real traffic
Total	**406**	**406 passed, 1 warning (29.17 s)**

Table V Automated test results by file (final run)

7.2 Security Demonstrations

The file test_security_demonstrations.py runs the real two-process setup with a live cloud server and records what actually crosses the network. Each demonstration is a normal test, so it fails if the behaviour changes. Table VI summarises them. D2, D3, D8 and D9 do not show bugs; they confirm leakage that is expected from the chosen design.

ID	What the test demonstrates
D1	The cloud never receives plaintext content, plaintext filenames or key material
D2	The same keyword always gives the same token, so repeated searches are recognisable
D3	The cloud sees the number of matches, the total index size, and which document ids are fetched after a search
D4	A tampered document ciphertext or a tampered index value fails loudly instead of returning wrong data
D5	Missing or corrupt KDF metadata on an existing vault fails closed
D6	A wrong passphrase is rejected before any upload or index replacement can happen
D7	Each upload re-fetches the whole growing corpus (static index rebuild cost is visible)
D8	A token captured before a rebuild still works afterwards and matches newly added files (no forward privacy)
D9	A two-keyword AND query is sent as two independent single-keyword searches

Table VI Security demonstrations in the test suite

The multi-user tests also check that account passwords are stored only as Argon2id hashes, that a wrong password and an unknown user get the same response, that sessions expire, that a regular user cannot reach admin routes, that one user cannot list, read, download or delete another user’s files, that the cloud rejects a wrong vault token, and that admin views contain no plaintext filenames, keywords

or keys.

7.3 Leakage Analysis

It is not correct to say that the cloud learns nothing. Table VII lists what the cloud can and cannot observe in our implementation, based on the code and the demonstrations above.

Information	Visible to the cloud?	Reason
File contents and filenames	No	Encrypted with AES-256-GCM under K_file
Search keywords	No	Only the token K1 is sent
Passphrase, master key, K_file, K_index, K2	No	Never leave the trusted client
Number of files, ciphertext sizes, upload times	Yes	GCM keeps the length; rows have size and timestamp
Total number of index rows	Yes	Equals the number of (keyword, file) pairs
Repeated searches (search pattern)	Yes	Same keyword gives the same deterministic token
Matching rows and result count (access pattern)	Yes	The cloud finds the rows and the client fetches the matching records
Which tokens belong to one AND query	Yes	The tokens arrive together for one request
Matches in files added after a search	Yes	No forward privacy: an old token still matches new files
Which user owns a vault	No	Vault ids are random; usernames stay on the client

Table VII What the untrusted cloud can and cannot observe

As explained in Section II, the search pattern and access pattern can be enough for inference attacks if the attacker has background knowledge about the documents [12], [13], and an attacker who can place files in the vault could learn queries quickly [14]. Our system does not defend against these attacks. Its protection is mainly against a curious storage provider that has no extra knowledge, and against other users and administrators of the same client.

VIII. LIMITATIONS AND FUTURE ENHANCEMENTS

While building and testing the system we noted these limitations:

- **Static index.** Every upload, delete and passphrase change rebuilds the whole index, which means downloading and decrypting every document (O(n) work). This is fine for a small vault but will become slow for large ones.
- **No forward privacy.** A token seen by the cloud keeps matching files added later. A forward-private scheme such as Σοφος [15] would fix this.
- **Leaky AND search.** Intersecting on the client reveals every keyword's own result count. A construction like OXT [11] would leak less.
- **Non-atomic passphrase change.** Re-encryption uses many separate requests and can leave a mixed state if interrupted. Envelope encryption (a random data key wrapped by the passphrase key) would make a passphrase change much cheaper and safer.
- **Trusted-client assumption.** Whoever controls the client machine can capture passphrases. The client database also stores each vault's cloud access token, which is enough to read ciphertext, though not to decrypt it.

- **Deployment.** Sessions are kept in memory, so restarting the client logs everyone out. The apps run on Flask's development server over plain HTTP on localhost; a real deployment needs HTTPS and the secure cookie flag.
- **Legacy namespace.** Unless the cloud is started with `CLOUD_ALLOW_LEGACY_VAULT=0`, requests without a vault id still reach the old single-user namespace without any token. This is kept only for backward compatibility with the earlier single-user version.
- **Search features.** Only exact keywords are supported; there is no stemming, substring, phrase or ranked search, and uploads are limited to text, text-layer PDFs and ZIPs up to 16 MB.
- **Key wiping.** Python cannot reliably erase key bytes from memory, so wiping is best-effort.

For future work we would like to add a forward-private dynamic index so that uploads do not need a full rebuild, use envelope encryption for fast passphrase changes, add padding or batching to hide result counts and file sizes, deploy behind HTTPS with persistent sessions, and measure performance properly on larger vaults.

IX. CONCLUSION

We built an encrypted cloud vault in which a trusted client encrypts everything and an untrusted cloud stores only ciphertext and an encrypted index. Keys come from a vault passphrase through Argon2id and HKDF-SHA256, files and filenames are protected with AES-256-GCM, and keyword search uses HMAC-SHA256 tokens and labels following the Π_{bas} construction. The current version adds multiple users, an administrator role with no access to keys, an audit log, and pages that show the real ciphertext and the real search messages. Our demo vault showed correct single-keyword and AND searches, isolation between users, and rejection of a one-bit change, and all 406 automated tests passed. The project also showed us that efficient searchable encryption always leaks something, so the leakage has to be written down honestly instead of hidden. The most important next step would be a forward-private, dynamic index.

REFERENCES

[1] D. X. Song, D. Wagner, and A. Perrig, "Practical techniques for searches on encrypted data," in *Proc. 2000 IEEE Symposium on Security and Privacy (S&P 2000)*, Berkeley, CA, USA, 2000, pp. 44–55, doi: 10.1109/SECPRI.2000.848445.

[2] R. Curtmola, J. Garay, S. Kamara, and R. Ostrovsky, "Searchable symmetric encryption: Improved definitions and efficient constructions," in *Proc. 13th ACM Conference on Computer and Communications Security (CCS '06)*, 2006, pp. 79–88.

[3] D. Cash, J. Jaeger, S. Jarecki, C. Jutla, H. Krawczyk, M.-C. Roşu, and M. Steiner, "Dynamic searchable encryption in very-large databases: Data structures and implementation," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2014, doi: 10.14722/ndss.2014.23264.

[4] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS PUB 197 (updated May 9, 2023), doi: 10.6028/NIST.FIPS.197-upd1.

[5] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nov. 2007.

[6] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, "Argon2 memory-hard function for password hashing and proof-of-work applications," RFC 9106, Internet Research Task Force, Sept. 2021.

[7] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," RFC 5869, Internet Engineering Task Force, May 2010.

[8] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, Internet Engineering Task Force, Feb. 1997.

[9] National Institute of Standards and Technology, "The Keyed-Hash Message Authentication Code (HMAC)," FIPS PUB 198-1, July 2008, doi: 10.6028/NIST.FIPS.198-1.

[10] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015, doi: 10.6028/NIST.FIPS.180-4.

[11] D. Cash, S. Jarecki, C. Jutla, H. Krawczyk, M.-C. Roşu, and M. Steiner, "Highly-scalable searchable symmetric encryption with support for Boolean queries," in *Advances in Cryptology – CRYPTO 2013*, LNCS 8042, Springer, 2013, pp. 353–373, doi: 10.1007/978-3-642-40041-4_20.

[12] M. S. Islam, M. Kuzu, and M. Kantarcioglu, "Access pattern disclosure on searchable encryption: Ramification, attack and mitigation," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2012.

[13] D. Cash, P. Grubbs, J. Perry, and T. Ristenpart, "Leakage-abuse attacks against searchable encryption," in *Proc. 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS '15)*, 2015, pp. 668–679, doi: 10.1145/2810103.2813700.

[14] Y. Zhang, J. Katz, and C. Papamanthou, "All your queries are belong to us: The power of file-injection attacks on searchable encryption," in *Proc. 25th USENIX Security Symposium*, Austin, TX, USA, 2016, pp. 707–720.

[15] R. Bost, "Σοφος: Forward secure searchable encryption," in *Proc. 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS '16)*, Vienna, Austria, 2016, pp. 1143–1154, doi: 10.1145/2976749.2978303.